Docker for IoT Systems Practice

Dockerfile — MQTT Subscriber

```
# Dockerfile for MQTT Subscriber service
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY mqtt_subscriber.py .

CMD ["python", "mqtt_subscriber.py"]
```

```
requirements.txt:
  paho-mqtt==2.0.0
```

```yaml
# docker-compose.yml

services:
  mosquitto:
    image: eclipse-mosquitto:2
    ports:
      - "1883:1883"
    volumes:
      - ./mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf

  api:
    build: ./api # use our Dockerfile in ./api folder
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=postgresql://iotuser:secret@postgres:5432/iotdb
      - MQTT_BROKER=mosquitto
    depends_on:
      - postgres
      - mosquitto
```

```yaml
  mqtt-subscriber:
    build: ./mqtt-subscriber
    environment:
      - MQTT_BROKER=mosquitto
      - INFLUX_URL=http://influxdb:8086
    depends_on:
      - mosquitto
      - influxdb

  postgres:
    image: postgres:16
    environment:
      - POSTGRES_USER=iotuser
      - POSTGRES_PASSWORD=secret
      - POSTGRES_DB=iotdb
    volumes:
      - postgres_data:/var/lib/postgresql/data   # persist data

  influxdb:
    image: influxdb:2.7
    ports:
      - "8086:8086"
    volumes:
      - influx_data:/var/lib/influxdb2

  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf
    depends_on:
      - api

volumes:
  postgres_data:
  influx_data:
```

--

Useful Docker Commands

```
# Build and start all services
docker compose up -d --build

# See all running containers
docker compose ps

# Execute a command inside a running container
docker compose exec api bash
docker compose exec postgres psql -U iotuser iotdb

# Restart a single service after code change
docker compose restart api

# Remove everything (containers + networks, keep volumes)
docker compose down
```

Summary

Docker solves the **"works on my machine"** problem and makes IoT backends reproducible, portable, and scalable.

Key concepts:

- **Dockerfile** defines how to build a service image
- **docker-compose.yml** defines the entire multi-service system
- **Service names** replace IP addresses for internal communication
- **Volumes** persist data across container restarts
- **Environment variables** keep configuration flexible and